

# Staff Bonus System — Audit Report

**Scope:** `mcm` (the portal app) + `csm-staging` Supabase project `reiogvwjunkdrbhewwv` **Date:** 2026-09-23  
**Method:** code read + live queries against staging. No code or data was modified. **Audience:** the engineers who will fix this, and whoever signs off on paying bonuses.

## 0. Headline

The bonus system does not implement the intended rule. Four defects compound, and each one inflates payouts independently:

| # | Defect                                                                                                                                   | Effect on the \$200-eligible / 10% / 4-staff example |
|---|------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| 1 | Pool is computed from <b>total sales</b> , not bonus-eligible sales                                                                      | \$100 pool instead of \$20                           |
| 2 | The team pool is an <b>accumulator that is never reset</b>                                                                               | grows without bound, day after day                   |
| 3 | The nightly distribution job <b>re-pays every team, every night, forever</b>                                                             | the same amount is written again each day            |
| 4 | The de-duplication key <b>does not work for the logged-in seller's row</b>                                                               | up to 8 copies of one payout observed                |
| 5 | The three bonus functions are <b>callable by anyone with the public anon key</b> , and the UI already calls two of them from the browser | the accumulator can be inflated at will, by anyone   |

On staging this currently writes **\$292,465.71 of individual bonus every single night**, the identical figure each night, 99% of it against sales teams that were closed months ago. Total accumulated: **\$32,259,545.68** across 425,345 rows.

The one thing the spec was most worried about — multiplying by staff count instead of dividing — is **not** a defect. The division is correct. Everything upstream of it is wrong.

## 1. Current implementation

The bonus system is **not implemented in the application**. It is three PL/pgSQL functions driven by three `pg_cron` jobs. The Next.js app only writes configuration and displays results.

## The flow, end to end



## The cron schedule (live, cron.job )

| jobid | name                               | schedule (UTC) | command                                            |
|-------|------------------------------------|----------------|----------------------------------------------------|
| 25    | update_bonus_totalsales_daily_11pm | 0 23 * * *     | SELECT public.update_bonus_totalsales();           |
| 27    | team_bonus_daily_1110pm_utc        | 10 23 * * *    | SELECT public.calculate_team_bonus_daily();        |
| 28    | distribute_individual_bonus_1115pm | 15 23 * * *    | SELECT public.distribute_individual_bonus_daily(); |

All three are `active = true` .

A fourth function, `public.trigger_distribute_individual_bonus()` , exists but is **attached to no trigger** — verified against `pg_trigger` . It is dead code that duplicates the distribution logic.

## 1b. The Settings questions, answered

| Question                                              | Answer                                                                                                                                                                     |
|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Where is the bonus percentage stored?                 | <code>bonus_config_history.value</code> , with <code>flat_percentage</code> selecting whether it means a percent or a flat dollar amount                                   |
| Is it clinic-specific?                                | Yes, per <code>location_id</code> . Every payload carries one                                                                                                              |
| Can it be changed from Settings?                      | Yes — the "Set limits" tab, <code>bonus/location/page.tsx:1562–1612</code>                                                                                                 |
| Is there a default value?                             | <b>No numeric default.</b> Only the <i>type</i> defaults, to the literal <code>'FLAT'</code> . <code>value</code> and <code>bonus_threshold</code> default to empty string |
| Is it validated?                                      | Barely. See below                                                                                                                                                          |
| Is it used by the backend calculation?                | Yes — <code>update_bonus_totalsales()</code> joins <code>bonus_config_history</code> on <code>effective_to IS NULL</code>                                                  |
| Can individual staff have different percentages?      | <b>No.</b> No per-staff rate exists anywhere in schema or code                                                                                                             |
| Is a single fixed clinic-level percentage the intent? | Yes, and that is what the schema models. The defect is in how the rate is <i>applied</i> , not where it is stored                                                          |

### Validation — what actually runs

**Percentage ( `value` ), `bonus/location/page.tsx:1577–1600` :** a plain text input. No `type="number"`, no `min`, `max` or `step`. It strips non-numerics with `replace(/[^\d-]/g, '')`, collapses extra decimal points, and clamps to 0-100 **only when the type is already PERCENTAGE**.

Two holes:

- **FLAT has no upper bound at all.**
- The clamp reads the type *at the moment of typing*. Type `250` while set to FLAT, then switch the selector to PERCENTAGE: the selector's `onChange` does not re-validate, so `250` is saved as a percentage. This is how the live `value = 200.00` PERCENTAGE config on location 15 became possible.

**Threshold ( `bonus_threshold` ), lines 1607-1612: no validation whatsoever.** Raw `e.target.value` is stored. Consequences at save (line 1625, `Number(row.bonus_threshold)`):

- `"-500"` is accepted and posted verbatim.
- `"abc"` becomes `NaN`, which `JSON.stringify` serialises as `null`, so **the threshold is silently wiped and the user gets a success toast**.

**Server side:** `app/api/bonuses/save/route.ts` inserts `value` and `bonus_threshold` straight through with no range, sign or type check.

**Stricter validators exist and are dead.** `handleSetLimitValueChange` (854-878) clamps with `Math.max(0, Math.min(100, num))` and rounds to 2dp; `handleSetLimitThresholdChange` (880-896) strips minus signs; `handleSetLimitSubmit` (791-852) has `Number.isNaN` guards with toasts. None of the three is reachable from any JSX. The live path is strictly weaker than the abandoned one.

## 2. Database structure

### `bonus_config_history` — where the percentage lives

| column                       | type                                             | notes                                                                           |
|------------------------------|--------------------------------------------------|---------------------------------------------------------------------------------|
| <code>id</code>              | integer PK                                       |                                                                                 |
| <code>location_id</code>     | integer                                          | FK → <code>Locations(id)</code> . <b>Nullable.</b>                              |
| <code>flat_percentage</code> | varchar(20) NOT NULL                             | free text; the code compares to <code>'FLAT'</code> / <code>'PERCENTAGE'</code> |
| <code>value</code>           | numeric(10,2) NOT NULL                           | the percentage, or the flat dollar amount                                       |
| <code>bonus_threshold</code> | numeric(10,2) NOT NULL                           | daily sales goal                                                                |
| <code>effective_from</code>  | date NOT NULL, default <code>CURRENT_DATE</code> |                                                                                 |
| <code>effective_to</code>    | date NULL                                        | NULL = the currently active config                                              |
| <code>created_at</code>      | timestamp <b>without</b> time zone               |                                                                                 |

No unique constraint, no check constraint. Nothing stops two open configs for one location, a negative percentage, or a 5000% percentage.

### `bonus` — the daily pool, one row per location per day

| column                                     | type          | notes                                                                                                                                       |
|--------------------------------------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>id</code>                            | integer PK    |                                                                                                                                             |
| <code>location_id</code>                   | integer       | two overlapping FKs to <code>Locations</code> : <code>fk_bonus_location</code> (ON DELETE CASCADE) and <code>fk_location</code> (no action) |
| <code>date</code>                          | date NOT NULL |                                                                                                                                             |
| <code>total_sales</code>                   | numeric       | all sales that day                                                                                                                          |
| <code>bonus_sales</code>                   | numeric       | bonus-eligible sales that day, or 0 if the threshold was missed                                                                             |
| <code>bonus_amount</code>                  | numeric(10,2) | <b>the pool. This is the wrong number — see §3.</b>                                                                                         |
| <code>bonus_eligibility</code>             | boolean       | threshold met                                                                                                                               |
| <code>bonus_config_history_id</code>       | integer       | FK → <code>bonus_config_history(id)</code> ON DELETE SET NULL                                                                               |
| <code>paid</code> , <code>paid_date</code> | boolean, date |                                                                                                                                             |

Unique: `bonus_location_date_unique (location_id, date)` . This one is correct and is the only thing keeping the daily roll-up idempotent.

### `sales_team` — who was selling

| column                                          | type                    | notes                                                     |
|-------------------------------------------------|-------------------------|-----------------------------------------------------------|
| <code>id</code>                                 | bigint PK               |                                                           |
| <code>location_id</code>                        | bigint NOT NULL         | FK → <code>Locations(id)</code> ON DELETE CASCADE         |
| <code>members</code>                            | <b>bigint[]</b>         | staff ids. An array, so <b>no foreign key is possible</b> |
| <code>auth_member</code>                        | uuid                    | FK → <code>auth.users(id)</code> <b>ON DELETE CASCADE</b> |
| <code>bonus_amount_overall</code>               | numeric(12,2) default 0 | <b>a running accumulator — see §3</b>                     |
| <code>bonus_amount_individual</code>            | numeric(12,2) default 0 | written nowhere; dead column                              |
| <code>valid_from</code> , <code>valid_to</code> | timestamptz             | <code>valid_to IS NULL</code> = the active team           |

No unique constraint on "one open team per location". No check that `members` staff actually work at `location_id`.

### `individual_bonus` — the payout record

| column                                     | type                                            | notes                                                      |
|--------------------------------------------|-------------------------------------------------|------------------------------------------------------------|
| <code>id</code>                            | bigint PK                                       |                                                            |
| <code>staff_id</code>                      | bigint NULL                                     | FK → <code>staff(id)</code> <b>ON DELETE SET NULL</b>      |
| <code>sales_team_id</code>                 | bigint NULL                                     | FK → <code>sales_team(id)</code> <b>ON DELETE SET NULL</b> |
| <code>bonus</code>                         | numeric(12,2) NOT NULL default 0                |                                                            |
| <code>bonus_date</code>                    | date NOT NULL default <code>CURRENT_DATE</code> |                                                            |
| <code>auth_member</code>                   | uuid NULL                                       | FK → <code>profiles(id)</code> ON DELETE SET NULL          |
| <code>paid</code> , <code>paid_date</code> | boolean NOT NULL, date                          |                                                            |

Unique: `individual_bonus_unique (staff_id, sales_team_id, bonus_date)`, built **NULLS DISTINCT** (verified: `pg_index.indnullsnotdistinct = false`). See §10.

### `staff`

`id`, `full_name`, `phone`, `created_at`, `location_id **bigint[]**`. No FK to `Locations` (an array column cannot carry one). Primary key only.

### `sales_history`

`total_price` is **double precision** — binary floating point, used for money. `return_qty` exists but no bonus function reads it.

What the database prevents

| Invalid state                       | Prevented?                                                                                                                                                                                                  |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Duplicate staff on the same sale    | <b>No.</b> <code>members</code> is a plain array, no uniqueness. (None present today.)                                                                                                                      |
| Duplicate bonus records             | <b>Partly.</b> Works for <code>staff_id</code> rows, fails entirely for <code>staff_id IS NULL</code> rows.                                                                                                 |
| Staff belonging to the wrong clinic | <b>No.</b> Array columns cannot have FKs; nothing validates membership. (None present today.)                                                                                                               |
| Invalid bonus percentage            | <b>No.</b> No CHECK. One live row has <code>PERCENTAGE value = 200.00</code> .                                                                                                                              |
| Negative amounts                    | <b>No.</b> No CHECK on <code>bonus</code> , <code>value</code> , <code>bonus_threshold</code> , <code>bonus_amount</code> .                                                                                 |
| Missing relationships               | <b>No.</b> <code>bonus.location_id</code> , <code>individual_bonus.staff_id</code> and <code>.sales_team_id</code> are all nullable, and two of the FKs are ON DELETE SET NULL, which manufactures orphans. |
| Transaction/atomicity               | <b>No.</b> See §8.                                                                                                                                                                                          |

3. Current calculation logic

Step 1 — the pool: `update_bonus_totalsales()`

```
Input      : all sales_history rows where date_sold::date = CURRENT_DATE
            joined to inventory → products (for bonus_eligible)
            joined to bonus_config_history where effective_to IS NULL
Threshold  : bonus_eligibility := SUM(total_price) >= cfg.bonus_threshold
Eligible   : bonus_sales := SUM(total_price) FILTER (products.bonus_eligible), else 0
Pool       : bonus_amount :=
            FLAT      → cfg.value
            PERCENTAGE → SUM(s.total_price) * (cfg.value / 100) ← TOTAL SALES
Write      : INSERT INTO bonus ... ON CONFLICT (location_id, date) DO UPDATE
```

The percentage is applied to `SUM(s.total_price)` — every sale — not to `bonus_sales`. The function computes `bonus_sales` correctly and then never uses it in the pool.

Verified against live data ( `bonus` joined to `bonus_config_history` ):

| date       | location | total_sales | bonus_sales | rate | bonus_amount (actual) | eligible × rate (intended) |
|------------|----------|-------------|-------------|------|-----------------------|----------------------------|
| 2026-09-04 | 18       | 1,419       | 300         | 10%  | 141.90                | 30.00                      |
| 2026-09-07 | 18       | 1,469       | 623         | 10%  | 146.90                | 62.30                      |
| 2026-09-04 | 6        | 2,014       | 1,189       | 10%  | 201.40                | 118.90                     |
| 2026-09-06 | 15       | 746         | 579         | 200% | 1,492.00              | 1,158.00                   |

Every row equals `total_sales × rate/100`. Not one equals `bonus_sales × rate/100`.

Step 2 — team share: `calculate_team_bonus_daily()`

```
team_be      := SUM(sales_history.total_price) for bonus-eligible products,
              for this location and date, GROUP BY orders.sales_team_id
team_weight  := team_be / bonus.bonus_sales          (0 if bonus_sales = 0)
team_bonus   := ROUND(bonus.bonus_amount * team_weight, 2)
sales_team.bonus_amount_overall := COALESCE(bonus_amount_overall, 0) + team_bonus
```

The weighting is sound: weights sum to 1 across teams, so the pool is split, not multiplied. The **write is not**. It is `+=` onto a column that is never zeroed. A team stays open until someone presses Reset in the POS; the oldest open team on staging has been open since **2025-11-19**, ten months, and now carries `bonus_amount_overall = 21,680.39`.

Step 3 — per staff: `distribute_individual_bonus_daily()`

```
FOR t IN SELECT * FROM sales_team          ← every team ever created, no date filter
  skip if bonus_amount_overall IS NULL or 0
  member_count := array_length(members, 1)    (1 if members IS NULL)
  if auth_member IS NOT NULL: member_count += 1
  per_member_bonus := ROUND(bonus_amount_overall / member_count, 2)
  INSERT one row per member, plus one row for auth_member, with bonus_date = CURRENT_DATE
  ON CONFLICT (staff_id, sales_team_id, bonus_date) DO UPDATE
```

The division itself is right. Everything around it is wrong:

- it iterates **all 4,190 teams**, not today's;
- it uses the **lifetime accumulator** as today's pool;
- it stamps `bonus_date = CURRENT_DATE` on teams that closed months ago.

Against the required formula

| Required                                            | Implemented                                              | Verdict                       |
|-----------------------------------------------------|----------------------------------------------------------|-------------------------------|
| <code>bonusPool = eligibleAmount × pct / 100</code> | <code>pool = totalSales × pct / 100</code>               | <b>Wrong</b>                  |
| <code>staffBonus = bonusPool / numberOfStaff</code> | <code>ROUND(lifetimeAccumulator / memberCount, 2)</code> | Divisor right, dividend wrong |

The worked example from the brief

Goal \$1,000 · total sale \$1,000 · eligible \$200 · rate 10% · 4 staff.

| Stage                                          | Intended                               | This system                               |
|------------------------------------------------|----------------------------------------|-------------------------------------------|
| Threshold met                                  | yes                                    | yes                                       |
| Pool                                           | $\$200 \times 10\% = \textbf{\$20.00}$ | $\$1,000 \times 10\% = \textbf{\$100.00}$ |
| Per staff (day 1)                              | \$5.00                                 | \$20.00                                   |
| Per staff (day 2, same sales, team still open) | \$5.00                                 | \$40.00                                   |
| Per staff (day 30)                             | \$5.00                                 | \$600.00                                  |

It never computes  $\$200 \times 10\% \times 4$ . It computes something worse.

## 4. What is working correctly

These were checked and are sound. They should be preserved by any fix.

1. **Per-staff division is correct.** `bonus_amount_overall / member_count`, never multiplied by it. `distribute_individual_bonus_daily()` and the dead trigger function both divide.
2. **Multi-team weighting is correct.** `team_weight = team_be / bonus_sales` in `calculate_team_bonus_daily()`; weights sum to 1, so two teams split one pool.
3. **Eligible-sales identification is correct.** `products.bonus_eligible` joined through `inventory` is the right path, and `bonus_sales` is computed correctly — it is simply ignored by the pool formula.
4. **Threshold gating works.** Below `bonus_threshold`, `bonus_eligibility = false`, `bonus_sales = 0` and `bonus_amount = 0`.
5. **The daily roll-up is idempotent.** `bonus_location_date_unique` plus `ON CONFLICT DO UPDATE` means re-running `update_bonus_totalsales()` is safe.
6. **Config history is append-only.** `/api/bonuses/save` closes the old row with `effective_to` and inserts a new one rather than updating in place, which is the right shape for \$12.
7. **Historical bonus rows are not retroactively rewritten**, because the roll-up filters `date_sold::date = CURRENT_DATE`.

### Staff-count scaling — traced

Because the divisor is right, pool conservation holds **within a single night** for a team whose accumulator happens to equal the day's pool. With pool = \$20:

| staff | per staff | distributed |
|-------|-----------|-------------|
| 1     | 20.00     | 20.00       |
| 2     | 10.00     | 20.00       |
| 4     | 5.00      | 20.00       |
| 10    | 2.00      | 20.00       |
| 20    | 1.00      | 20.00       |

All exact, no rounding drift, because 20 divides evenly. The counts are not the problem.

Note one trap: `member_count = array_length(members,1) + 1` when `auth_member` is set. The logged-in seller is counted as an extra head. If the UI already includes that person in `members`, they are counted twice and everyone's share shrinks. **Whether the POS includes the seller in `members` is not verified from the existing implementation** — it depends on `components/POS/PosFields.tsx`.

---

## 5. What is incorrect

---

### 5.1 Pool uses total sales, not bonus-eligible sales — CRITICAL

`public.update_bonus_totalsales()`, the `bonus_amount` CASE expression. `SUM(s.total_price) * (cfg.value / 100)` must be `bonus_sales * (cfg.value / 100)`. Overpays by `total_sales / bonus_sales`; observed up to **4.73x** (location 18, 2026-09-04).

### 5.2 Team pool accumulates forever — CRITICAL

`public.calculate_team_bonus_daily()`: `SET bonus_amount_overall = COALESCE(bonus_amount_overall, 0) + team_bonus`. `sales_team` rows persist until a POS user presses Reset. Oldest open team: 2025-11-19, accumulator 21,680.39. Today's bonus is therefore the sum of every bonus since the team opened.

### 5.3 Distribution has no date filter — CRITICAL

`public.distribute_individual_bonus_daily()`: `FOR t IN SELECT * FROM public.sales_team`. It pays **every team that ever existed**, every night, dated today.

Measured, per night, identical on every one of the last 6 nights:

| bonus_date | rows written | of which for closed teams | teams touched | total written |
|------------|--------------|---------------------------|---------------|---------------|
| 2026-09-22 | 2,189        | 2,166                     | 788           | \$292,465.71  |
| 2026-09-21 | 2,189        | 2,166                     | 788           | \$292,465.71  |
| 2026-09-20 | 2,189        | 2,166                     | 788           | \$292,465.71  |

Worked example — team 3217 (location 27, 2 members + `auth_member`), `bonus_amount_overall` frozen at 9,012.20, pays **\$9,012.21 to 3 people every single day** regardless of that day's sales.

### 5.4 Re-running the team job double-pays — HIGH

`calculate_team_bonus_daily()` has no idempotency guard: no `ON CONFLICT`, no "already processed" marker, no date column on `sales_team`. A manual re-run, a cron retry, or a `pg_cron` overlap adds `team_bonus` a second time, permanently.

### 5.5 `bonus.paid` and the individual rows disagree — HIGH

`app/api/bonuses/update-paid/route.ts` finds teams with `.eq('valid_from', date)` — i.e. teams **created** on the bonus date. A team opened in November and still open has `valid_from` in November, so marking any later date paid matches no team and **no `individual_bonus` row is updated**. Live counts: `bonus.paid` = 6 rows, but `individual_bonus.paid` = 23 rows, out of 425,345.

## 5.6 The non-config branch of `/api/bonuses/save` writes columns that do not exist — MEDIUM (latent)

`app/api/bonuses/save/route.ts` lines ~137–195 update `bonus` with `flat_percentage`, `value` and `bonus_threshold`. The `bonus` table has none of those columns. Any call would 500. Currently unreachable: every caller in `bonus/location/page.tsx` passes `configOnly: true`.

## 5.7 Eligibility comparison is inconsistent — LOW

SQL uses `>= (SUM(total_price) >= cfg.bonus_threshold)`; the API recompute at `app/api/bonuses/save/route.ts` uses `Number(totalSales) > Number(threshAmount)`. Exactly hitting the goal is eligible in SQL, not eligible in the API.

## 5.8 Dead code — LOW

`public.trigger_distribute_individual_bonus()` — no trigger references it.  
`sales_team.bonus_amount_individual` — never written.

---

## 5.9 The same pool bug is duplicated in the browser — HIGH

`app/[locale]/(root)/(childroot)/bonus/location/page.tsx:466–497`, `computeBonusAmount()`. Line 467 reads `total_sales`; line 484 computes `amount = total * (pct / 100)`. Identical defect to §5.1, in a second implementation. The page reads `bonus_sales` (lines 1071–1072, 1698) only to display it, never to calculate. This computed figure is shown in preference to the stored `bonus_amount` for edited rows (1172–1176) and as a fallback elsewhere (1180–1181, 1714), so the screen and the database can disagree, and both are wrong the same way.

Two further divergences between the browser copy and the SQL:

- For PERCENTAGE the threshold is an all-or-nothing gate (`if (total <= limit) amount = 0`, lines 488–489); for FLAT it additionally caps at `total - limit` (490–493). The SQL has no such FLAT cap.
- Line 480 compares `type === 'FLAT'` case-sensitively while every display path upper-cases first. A stored `'Flat'` would be computed as a percentage.

## 5.10 The bonus page shows every clinic — HIGH

`app/[locale]/(root)/(childroot)/bonus/location/fetch.ts:5–8` fetches the whole `Locations` table with no user scoping. The user-scoped helper `fetchLocations(userId)` in `utils/supabase/data_services/data_services.tsx:76–89`, which filters by `user_locations`, exists and is **not used here** — the bonus page shadows the name with its own unscoped version. The individual page is the same: `individual_bonus`, `Locations` and `staff` are all fetched without a user filter. Location filtering is a client-side convenience only.

## 6. Potential bugs

---

- Business day is a UTC day.** DB `TimeZone = UTC`; the jobs use `CURRENT_DATE` and `date_sold::date`. Clinics are US Central. A sale at 7pm Central lands in the **next** UTC day's bucket. The bonus "day" effectively runs 7pm–7pm local. **HIGH**

2. **The job runs before the day ends.** 23:00 UTC = 6pm Central. Sales after 6pm are handled only by the following night's run, under the following day's config. **MEDIUM**
3. **Floating-point money.** `sales_history.total_price` is `double precision`, so `SUM(total_price)` carries binary rounding error into `total_sales`, `bonus_sales` and the pool before any `ROUND` is applied. **MEDIUM**
4. **Two open configs break the nightly job.** `update_bonus_totalsales()` groups by the config columns; two rows with `effective_to IS NULL` for one location produce two result rows for the same `(location_id, date)`, and `ON CONFLICT DO UPDATE` then raises *"cannot affect row a second time"*, aborting the whole run for **every** location. Nothing prevents two open configs — and `app/api/bonuses/active-configs/route.ts` contains explicit de-duplication logic, which shows the team already knows it happens. None present today. **HIGH**
5. **Locations with no config are silently skipped.** `update_bonus_totalsales()` inner-joins `bonus_config_history`. No config, no bonus row, no error, no warning. **MEDIUM**
6. **UTC date for "today" in the API.** `new Date().toISOString().slice(0,10)` in `/api/bonuses/save` and `/api/bonuses/update-paid` — after 7pm Central this is tomorrow's date, so a config saved in the evening is stamped a day ahead. **MEDIUM**
7. **Deleting a staff row silently redirects their money.** `individual_bonus.staff_id` is `ON DELETE SET NULL`, which both orphans the payout and drops it into the NULL bucket where the unique index no longer applies. **MEDIUM**
8. **Deleting an auth user cascades away sales teams.** `sales_team.auth_member` is `ON DELETE CASCADE`, which deletes the **team row**, not just the link, and then `individual_bonus.sales_team_id` goes NULL. Bonus history is destroyed by deleting a user account. **HIGH**

---

## 7. Security / authorization issues

---

### 7.1 Every bonus table is world-writable with the public anon key — CRITICAL

RLS is *enabled* on `bonus`, `bonus_config_history`, `individual_bonus`, `sales_team` and `staff`, but every policy is `USING (true)` / `WITH CHECK (true)`, and the policies are granted to **PUBLIC** (`pg_policy.polroles = {0}`), which includes the `anon` role. Table grants confirm `anon` holds `SELECT, INSERT, UPDATE, DELETE, TRUNCATE` on all five.

The anon key ships in the browser bundle of every Next.js page. With it, anyone on the internet can:

- read every clinic's sales totals and every staff member's earnings;
- change any clinic's bonus percentage (`bonus_config_history`);
- mark any bonus paid or unpaid (`bonus.paid`, `individual_bonus.paid`);
- insert bonus rows for themselves;
- `TRUNCATE` the entire bonus history.

Policy inventory:

| table                | policies                           | all true ?                                                                                                                     | granted to                |
|----------------------|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| bonus                | 4<br>(SELECT/INSERT/UPDATE/DELETE) | yes                                                                                                                            | PUBLIC                    |
| bonus_config_history | 4                                  | yes                                                                                                                            | PUBLIC                    |
| individual_bonus     | 4                                  | yes                                                                                                                            | PUBLIC                    |
| sales_team           | 13 (9 duplicates)                  | yes, except one INSERT policy<br>auth.uid() = auth_member<br>which is neutralised by the<br>permissive true policies beside it | PUBLIC +<br>authenticated |
| staff                | 5                                  | yes                                                                                                                            | PUBLIC +<br>authenticated |

**Clinic A can read and modify clinic B's bonus data.** There is no tenant predicate anywhere — not in RLS, not in the API routes. Nothing in the data model ties a request to a permitted location set.

## 7.2 No role check on any bonus API route — HIGH

| route                                              | authentication            | role / permission check |
|----------------------------------------------------|---------------------------|-------------------------|
| POST /api/bonuses/save (sets the percentage)       | none                      | none                    |
| POST /api/bonuses/update-paid (marks bonuses paid) | none                      | none                    |
| GET /api/bonuses/active-configs                    | none                      | none                    |
| GET /api/bonuses/config-query                      | none                      | none                    |
| POST /api/sales-team                               | getUser() , 401 if absent | none                    |
| POST /api/sales-team/reset                         | getUser() , 401 if absent | none                    |
| POST /api/orders                                   | none                      | none                    |

The two save/pay routes do not call `supabase.auth.getUser()` at all. They rely on RLS, and RLS is `true`. The two sales-team routes authenticate but never check that the caller may act on the `location_id` they passed, so any logged-in user can reset or re-staff any clinic's team.

### 7.3 Who can do what — as built

| action                         | intended                                          | actual                                                                                               |
|--------------------------------|---------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Configure the bonus percentage | clinic admin                                      | anyone with the anon key                                                                             |
| Mark an item bonus-eligible    | admin<br>( <code>products.bonus_eligible</code> ) | <code>products</code> RLS not audited in depth; <b>not verified from the existing implementation</b> |
| Assign staff to a sale         | POS user at that location                         | any authenticated user, any location                                                                 |
| Approve / mark a bonus paid    | admin                                             | anyone with the anon key                                                                             |
| Modify or reverse a bonus      | admin                                             | anyone with the anon key                                                                             |
| Run the bonus calculation      | the nightly job only                              | anyone with the anon key (see 7.4)                                                                   |

### 7.4 The bonus functions are executable by the public anon role — CRITICAL

`pg_proc.proacl` for all three functions reads `anon=X/postgres | authenticated=X/postgres`. They are **not** `SECURITY DEFINER`, but the anon role already has full DML on every table they touch (7.1), so that changes nothing.

Supabase exposes any function in the `public` schema as a PostgREST RPC endpoint. So `POST /rest/v1/rpc/calculate_team_bonus_daily` with the published anon key increments `bonus_amount_overall` for every team by another day's worth — and because that function is a bare `+=` with no idempotency guard (§5.4), calling it in a loop inflates every clinic's bonus pool without limit.

`distribute_individual_bonus_daily` then writes those inflated figures into `individual_bonus`, creating a fresh duplicate row per call for every seller (§10.1).

This is not hypothetical: the application already does it from the browser.

`app/[locale]/(root)/(childroot)/bonus/individual/page.tsx:536–554` calls `supabase.rpc('calculate_team_bonus_daily')` followed by `supabase.rpc('distribute_individual_bonus_daily')` on a user action. Every such click adds an extra, unscheduled increment on top of the nightly cron run.

### 7.5 UI gating — verified

- There is **no permission check inside either bonus page**. Neither file imports a permission hook, reads a role, or redirects.
- Gating comes only from `app/[locale]/(root)/(childroot)/layout.tsx`, which wraps the route group in `hoc/withAuthorization.tsx`. That HOC looks the path up in `routeList` (`components/Sidebar/constant.tsx`) and denies anything it cannot find. **`routeList` contains no `/bonus` entry** — only an unused constant `BONUS: "/bonus"` at line 67. So `findRouteByPath('/bonus/location', routeList)` returns null and every non-super-admin is refused.
- Net effect: **super admin only**, by accident rather than design. The explicit special case at `withAuthorization.tsx:74–79` that grants bonus access to holders of the `control` permission is dead code that can never fire.
- This is UI-only. It does not protect the data, which is reachable with the anon key (7.1, 7.4).

## 7.6 The approval path writes from the browser — HIGH

`bonus/individual/page.tsx:388-412` marks a bonus paid by calling `update_content_service` on `individual_bonus` **directly from the client**, not through any API route. There is no server route in that path at all, so there is nothing to add an authorization check to.

Worse, `handlePay` cannot tell success from failure. Lines 398-405 have two branches with identical bodies, and the second is commented *"fallback: still mark as paid if service didn't return rows but no error thrown"*. An RLS-blocked update returns zero rows without throwing, so the row renders as Paid and the button disables while the database is unchanged. Line 410 is an explicit `// TODO` to surface the error.

---

## 8. Data integrity issues

1. **No transaction or atomicity anywhere.** Each of the three functions is its own statement; there is no wrapping transaction across pool → team → distribution. If job 27 succeeds and job 28 fails, teams carry an incremented accumulator with no payout rows, and the next night's run silently pays both days as one. **HIGH**
  2. `app/api/orders/route.ts` **is not atomic.** Order, sales history, transaction history, credit audit and location balance are five separate awaited calls with no rollback. A failure after the order insert leaves a half-written sale that the bonus roll-up will still count. **HIGH**
  3. **Nullable everything.** `bonus.location_id`, `individual_bonus.staff_id`, `individual_bonus.sales_team_id` and `bonus_config_history.location_id` are all nullable. A payout row can exist attached to nobody, for no team.
  4. **Two conflicting FKs on `bonus.location_id`** — `fk_bonus_location` (ON DELETE CASCADE) and `fk_location` (NO ACTION). Behaviour on location delete depends on which the planner applies. Deleting a location can silently erase its bonus history.
  5. **No CHECK constraints.** Nothing enforces `bonus >= 0`, `0 <= value <= 100` for PERCENTAGE, `bonus_threshold >= 0`, or `valid_to > valid_from`. One live config has `value = 200.00` with mode PERCENTAGE, producing a pool of **twice** the day's total sales.
  6. `members bigint[]` and `staff.location_id bigint[]` **cannot be referentially enforced.** Today's data happens to be clean — 0 teams contain a staff member not assigned to that location, 0 teams contain a duplicate staff id — but this is luck, not a constraint.
  7. **No audit trail on approval.** `bonus.paid` / `individual_bonus.paid` record *that* and *when*, never *who*. There is no approver column anywhere.
- 

## 9. Rounding issues

The system uses `numeric` at the payout layer and `double precision` at the sales layer.

- `sales_history.total_price` is `double precision`. Sums feeding `total_sales` and `bonus_sales` carry binary floating-point error.
- `bonus.bonus_amount` is `numeric(10,2)`, `individual_bonus.bonus` is `numeric(12,2)`, `sales_team.bonus_amount_overall` is `numeric(12,2)`. Good types.

- Both distribution functions use `ROUND(x / n, 2)` — **round-half-away-from-zero, applied independently to each share, with no remainder allocation.**

There is **no largest-remainder step**. The sum of the shares does not equal the pool.

The brief's example, `$10 / 3`, produces `3.33 + 3.33 + 3.33 = 9.99` — **one cent lost**. It does **not** produce the required `3.34 + 3.33 + 3.33 = 10.00`.

And the error runs both ways. Live proof, team 3217: pool 9,012.20, 3 members, `ROUND(9012.20/3, 2) = 3004.07`, `× 3 = 9012.21` — **one cent created from nothing**, every night, on one team alone.

| pool     | staff | per staff | distributed | drift                 |
|----------|-------|-----------|-------------|-----------------------|
| 10.00    | 3     | 3.33      | 9.99        | −0.01                 |
| 20.00    | 3     | 6.67      | 20.01       | +0.01                 |
| 9,012.20 | 3     | 3,004.07  | 9,012.21    | +0.01 (observed live) |
| 20.00    | 4     | 5.00      | 20.00       | 0.00                  |

Client-side formatting was delegated to a separate pass and is **not verified from the existing implementation**.

## 10. Duplicate-payment risks

### 10.1 The unique index does not cover the seller's own row — CRITICAL

`individual_bonus (staff_id, sales_team_id, bonus_date)` is built **NULLS DISTINCT** (`pg_index.indnullsnotdistinct = false`). Both distribution functions insert the `auth_member` row with `staff_id = NULL`:

```
INSERT INTO public.individual_bonus (staff_id, sales_team_id, bonus, bonus_date, created_at,
auth_member)
VALUES (NULL, t.id, per_member_bonus, target_date, NOW(), t.auth_member)
ON CONFLICT (staff_id, sales_team_id, bonus_date) DO UPDATE ...
```

In PostgreSQL two NULLs are never equal, so **the conflict never fires** and every execution inserts a brand-new row. `ON CONFLICT DO UPDATE` is decorative for these rows.

Measured live:

| metric                                                                | value    |
|-----------------------------------------------------------------------|----------|
| rows with <code>staff_id IS NULL</code>                               | 145,993  |
| duplicated <code>(sales_team_id, bonus_date)</code> groups among them | 1,462    |
| <b>maximum copies of a single payout</b>                              | <b>8</b> |

Eight identical bonus rows for the same person, same team, same day.

## 10.2 No idempotency on the team accumulator — HIGH

`calculate_team_bonus_daily()` is `+=` with no guard (§5.4). Two runs, double the pool.

## 10.3 No idempotency key on order creation — MEDIUM

`app/api/orders/route.ts` accepts a POST and inserts. No `Idempotency-Key`, no client-supplied uuid, no dedupe on `(patient, location, cart, timestamp)`. A double-click or a network retry creates a second order, second `sales_history` rows, and therefore a second contribution to the day's eligible sales.

## 10.4 Race conditions — MEDIUM

`app/api/orders/route.ts` does read-then-write on the active sales team: `SELECT ... WHERE valid_to IS NULL LIMIT 1`, then `INSERT` if empty. Two concurrent checkouts at one location can both see no team and both insert one. Nothing in the schema prevents two open teams. The `pg_cron` jobs are fixed 5- and 10-minute offsets, not chained, so a slow job 25 can still be running when job 27 starts.

# 11. Refund / cancellation issues

There is no reversal path. None.

| lifecycle event    | handled?                      | evidence                                                                                                 |
|--------------------|-------------------------------|----------------------------------------------------------------------------------------------------------|
| Completed          | yes                           | <code>POST /api/orders</code>                                                                            |
| Cancelled          | <b>no such concept</b>        | <code>orders</code> has no status column                                                                 |
| Voided             | <b>no such concept</b>        | no void route, no flag                                                                                   |
| Refunded           | <b>not reflected in bonus</b> | <code>returns</code> table + <code>sales_history.return_qty</code> exist; no bonus function reads either |
| Partially refunded | <b>not reflected in bonus</b> | same                                                                                                     |
| Hard delete        | <b>not reflected in bonus</b> | <code>app/api/orders/delete/route.ts</code>                                                              |

Details:

- `orders` has no `status`, `voided_at` or `cancelled_at` column. A sale can only be **deleted**.
- `app/api/orders/delete/route.ts` deletes `discounts`, `returns`, `sales_history`, `transaction_history` and `orders`. It never touches `bonus`, `sales_team.bonus_amount_overall` or `individual_bonus`. Because `update_bonus_totalsales()` only ever processes `CURRENT_DATE`, deleting yesterday's order leaves yesterday's inflated pool permanently in place.
- `sales_history.return_qty` and the `returns` table are ignored by every bonus function. `SUM(s.total_price)` is the gross figure. A fully refunded sale still earns its bonus.
- Because `bonus_amount_overall` only ever increases (§5.2), a reversal is not even expressible in the current model without going negative on a column that has no CHECK allowing or forbidding it.
- Nothing prevents reversing a bonus that has already been marked paid.

Staging currently holds 0 rows in `returns` and 0 `sales_history` rows with `return_qty > 0`, so **the refund path has never been exercised** — the gap is structural, not yet realised.

---

## 12. Historical calculation issues

---

The scenario in the brief — 10% on Sept 20, admin changes to 15% on Sept 21, Sept 20 must stay at 10% — **holds today, but by accident, and it is fragile.**

What works:

- `/api/bonuses/save` (configOnly branch) closes the current row with `effective_to = today` and inserts a new row with `effective_from = today`, so the rate history is preserved.
- `bonus.bonus_config_history_id` records which config produced each day's row.
- `update_bonus_totalsales()` filters `date_sold::date = CURRENT_DATE` and upserts on `(location_id, date)`, so it never revisits a past date. Sept 20's row is not recomputed.

What is fragile or wrong:

1. **The roll-up joins `effective_to IS NULL`, not the config effective on the sale date.** It works only because it is always run on the same day as the sales. Any backfill, replay or manual re-run of `update_bonus_totalsales()` would reprice **every** processed day at **today's** rate. There is no date-ranged join available for that.
  2. **Same-day changes silently reprice the current day.** Change the rate at 2pm and the 23:00 job uses the new rate for the whole day, including the morning's sales. The old config's `effective_to` and the new one's `effective_from` are both today, so the day is ambiguous.
  3. **individual\_bonus has no rate provenance.** It stores an amount, a date and a team. It has no link to `bonus` or to `bonus_config_history`, so a payout row cannot be traced to the rate that produced it. Reconstructing why a staff member was paid \$X requires joining through `sales_team` and guessing the date.
  4. **The accumulator destroys historical meaning anyway.** Because `bonus_amount_overall` is a lifetime running total (§5.2) and is redistributed nightly (§5.3), an `individual_bonus` row dated Sept 20 does not correspond to Sept 20's sales at any rate.
  5. `/api/bonuses/active-configs` reads `effective_to IS NULL` for the no-date case and `effective_from <= date AND (effective_to IS NULL OR effective_to > date)` for the dated case. The dated query is correct. The SQL function does not use it.
-

## 13. Exact files, functions and models involved

Database — `csm-staging` ( `reiogvwjunkdrbhewwv` )

| object                                                    | kind                  | role                                                                   |
|-----------------------------------------------------------|-----------------------|------------------------------------------------------------------------|
| <code>public.update_bonus_totalsales()</code>             | function, cron job 25 | computes the daily pool. <b>Defect 5.1</b>                             |
| <code>public.calculate_team_bonus_daily()</code>          | function, cron job 27 | splits the pool across teams. <b>Defects 5.2, 5.4</b>                  |
| <code>public.distribute_individual_bonus_daily()</code>   | function, cron job 28 | per-staff payout. <b>Defects 5.3, 9, 10.1</b>                          |
| <code>public.trigger_distribute_individual_bonus()</code> | function              | dead, attached to no trigger                                           |
| <code>public.bonus</code>                                 | table                 | daily pool per location                                                |
| <code>public.bonus_config_history</code>                  | table                 | the percentage, threshold, mode                                        |
| <code>public.sales_team</code>                            | table                 | who sold; holds the accumulator                                        |
| <code>public.individual_bonus</code>                      | table                 | the payout records                                                     |
| <code>public.staff</code>                                 | table                 | staff roster, <code>location_id</code> bigint[]                        |
| <code>public.sales_history</code>                         | table                 | <code>total_price</code> double precision ,<br><code>return_qty</code> |
| <code>public.orders</code>                                | table                 | <code>sales_team_id</code> links a sale to a team                      |
| <code>public.products.bonus_eligible</code>               | column                | flags eligible items (1,212 of 1,626)                                  |
| <code>public.returns</code>                               | table                 | never read by the bonus system                                         |
| <code>individual_bonus_unique</code>                      | index                 | NULLS DISTINCT. <b>Defect 10.1</b>                                     |
| <code>bonus_location_date_unique</code>                   | index                 | correct                                                                |

## Application — mcm

| file                                                                  | role                                                                                           |
|-----------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| app/api/bonuses/save/route.ts                                         | writes <code>bonus_config_history</code> (configOnly). No auth. Dead broken branch at ~137–195 |
| app/api/bonuses/update-paid/route.ts                                  | approval/payment. No auth. <b>Defect 5.5</b>                                                   |
| app/api/bonuses/active-configs/route.ts                               | reads configs; has ad-hoc de-dup                                                               |
| app/api/bonuses/config-query/route.ts                                 | per-location/date config lookup                                                                |
| app/api/sales-team/route.ts                                           | creates a team, closes the previous. Authenticated, not authorised                             |
| app/api/sales-team/reset/route.ts                                     | closes the open team. Authenticated, not authorised                                            |
| app/api/orders/route.ts                                               | the sale. Writes <code>orders.sales_team_id</code> . Non-atomic, no idempotency                |
| app/api/orders/delete/route.ts                                        | hard-deletes a sale. <b>No bonus reversal</b>                                                  |
| app/[locale]/(root)/(childroot)/bonus/location/page.tsx               | settings + approval UI                                                                         |
| app/[locale]/(root)/(childroot)/bonus/individual/page.tsx             | per-staff view                                                                                 |
| app/[locale]/(root)/(childroot)/bonus/location/fetch.ts               | data loader                                                                                    |
| components/BonusSummaryCards.tsx ,<br>components/BonusFilterSheet.tsx | display                                                                                        |
| components/P0S/PosFields.tsx                                          | staff selection; calls sales-team create/reset                                                 |

## 14. Severity summary

| #  | Finding                                                                                                        | Severity | Where                                                                           |
|----|----------------------------------------------------------------------------------------------------------------|----------|---------------------------------------------------------------------------------|
| 1  | Pool computed on total sales, not eligible sales                                                               | Critical | <code>update_bonus_totalsales()</code>                                          |
| 2  | <code>bonus_amount_overall</code> accumulates forever, never reset                                             | Critical | <code>calculate_team_bonus_daily()</code>                                       |
| 3  | Distribution has no date filter; pays all 4,190 teams nightly                                                  | Critical | <code>distribute_individual_bonus_daily()</code>                                |
| 4  | Unique index NULLS DISTINCT; seller rows duplicate (8 copies seen)                                             | Critical | <code>individual_bonus_unique</code>                                            |
| 5  | Anon key has full read/write/TRUNCATE on all bonus tables; no tenant isolation                                 | Critical | RLS policies, table grants                                                      |
| 5b | <code>anon / authenticated</code> hold EXECUTE on all three bonus functions; the UI calls two from the browser | Critical | <code>pg_proc.proacl</code> ,<br><code>bonus/individual/page.tsx:536-554</code> |
| 6  | No role check on the configure and approve endpoints                                                           | High     | <code>bonuses/save</code> , <code>bonuses/update-paid</code>                    |
| 6b | Approval writes <code>individual_bonus</code> straight from the browser, and cannot detect failure             | High     | <code>bonus/individual/page.tsx:388-412</code>                                  |
| 6c | Client <code>computeBonusAmount</code> repeats the total-vs-eligible bug                                       | High     | <code>bonus/location/page.tsx:466-497</code>                                    |
| 6d | Bonus page lists every clinic; user-scoped loader exists but is unused                                         | High     | <code>bonus/location/fetch.ts:5-8</code>                                        |
| 6e | <code>bonus_threshold</code> input has no validation; non-numeric silently wipes it to null                    | High     | <code>bonus/location/page.tsx:1607-1612</code>                                  |
| 6f | Percentage clamp bypassed by switching FLAT to PERCENTAGE after typing                                         | Medium   | <code>bonus/location/page.tsx:1577-1600</code>                                  |
| 7  | No refund / void / cancellation reversal                                                                       | High     | whole system                                                                    |
| 8  | Team job not idempotent; re-run double-pays                                                                    | High     | <code>calculate_team_bonus_daily()</code>                                       |
| 9  | Paid flag does not reach <code>individual_bonus</code>                                                         | High     | <code>bonuses/update-paid</code>                                                |
| 10 | No transaction across the three stages                                                                         | High     | cron jobs 25/27/28                                                              |
| 11 | Deleting an auth user cascade-deletes sales teams                                                              | High     | <code>sales_team_auth_member_fkey</code>                                        |
| 12 | Two open configs abort the entire nightly run                                                                  | High     | <code>update_bonus_totalsales()</code>                                          |
| 13 | Business day is a UTC day; job runs at 6pm local                                                               | High     | cron schedule, <code>CURRENT_DATE</code>                                        |
| 14 | Rounding loses or creates cents; no remainder allocation                                                       | Medium   | both distribution functions                                                     |
| 15 | <code>total_price</code> is <code>double precision</code>                                                      | Medium   | <code>sales_history</code>                                                      |
| 16 | No CHECK constraints; a 200% rate is live                                                                      | Medium   | all four tables                                                                 |

| #  | Finding                                                                     | Severity | Where                                                           |
|----|-----------------------------------------------------------------------------|----------|-----------------------------------------------------------------|
| 17 | No idempotency key on order creation                                        | Medium   | orders/route.ts                                                 |
| 18 | Race on active-team read-then-write                                         | Medium   | orders/route.ts                                                 |
| 19 | ON DELETE SET NULL orphans payouts into the un-deduplicated NULL bucket     | Medium   | individual_bonus FKs                                            |
| 20 | Locations without a config silently produce no bonus                        | Medium   | update_bonus_totalsales()                                       |
| 21 | UTC <code>toISOString().slice(0,10)</code> used as "today"                  | Medium   | bonuses/save , bonuses/update-paid                              |
| 22 | No approver identity recorded                                               | Medium   | bonus , individual_bonus                                        |
| 23 | Dead broken branch writing non-existent columns                             | Medium   | bonuses/save ~137-195                                           |
| 24 | <code>&gt;=</code> in SQL vs <code>&gt;</code> in the API for the threshold | Low      | update_bonus_totalsales() vs bonuses/save                       |
| 25 | Dead function and dead column                                               | Low      | trigger_distribute_individual_bonus() , bonus_amount_individual |

## 15. Recommended changes

Not implemented — this is an audit. Listed in the order I would do them.

### Stop the bleeding (today)

1. **Disable cron jobs 27 and 28.** They are writing \$292k of bad payout data every night. Job 25 (`update_bonus_totalsales`) can stay; it is idempotent and only writes `bonus`.
2. **REVOKE EXECUTE ON FUNCTION `calculate_team_bonus_daily`, `distribute_individual_bonus_daily`, `update_bonus_totalsales` FROM anon, authenticated.** They only need to run from `pg_cron`. Then remove the two `supabase.rpc(...)` calls at `bonus/individual/page.tsx:536-554`; the UI must not be able to trigger a payout run.
3. **Lock down RLS.** Replace all `USING (true)` policies on `bonus`, `bonus_config_history`, `individual_bonus`, `sales_team` and `staff` with location-scoped policies keyed off `user_locations`, and `REVOKE ALL ... FROM anon` on all five. This is exploitable right now with a key that is published in the page source.
4. **Quarantine the existing data.** 425,345 `individual_bonus` rows and every `sales_team.bonus_amount_overall` are untrustworthy. Do not pay from them. They need a recomputation from `sales_history`, not a patch.

### Fix the calculation

5. In `update_bonus_totalsales()`, change the PERCENTAGE branch to `bonus_sales * (cfg.value / 100)`. One line. This is the defect the brief asked about.
6. Replace `sales_team.bonus_amount_overall +=` with a **per-day** figure. Add a `team_bonus_daily(sales_team_id, date, amount)` table with a unique key on `(sales_team_id, date)`

and upsert into it, leaving `sales_team` as reference data.

7. Give `distribute_individual_bonus_daily()` a date filter so it only processes teams with activity on the target date, and drive the per-staff amount from the new daily table.
8. Rebuild `individual_bonus_unique` with `NULLS NOT DISTINCT`, or better: stop using NULL for the seller. Add a `profile_id uuid` column, make the key `(sales_team_id, bonus_date, COALESCE(staff_id, -1), COALESCE(profile_id, ...))`, and de-duplicate the existing rows first.
9. Add largest-remainder allocation: compute `floor(pool*100/n)` cents for everyone, then distribute the leftover cents one each, deterministically ordered. `$10/3` must give `3.34 + 3.33 + 3.33`.
10. Wrap stages 1–3 in one function inside one transaction, and make it idempotent on `(location_id, date)` so a re-run is a no-op.

## Fix the model

11. Change `sales_history.total_price` to `numeric(12,2)`.
12. Add CHECK constraints: `bonus >= 0`; `value >= 0`; `value <= 100` when `flat_percentage = 'PERCENTAGE'`; `bonus_threshold >= 0`; `valid_to > valid_from`. Fix the live 200% config first.
13. Add a partial unique index `ON bonus_config_history (location_id) WHERE effective_to IS NULL` and one `ON sales_team (location_id) WHERE valid_to IS NULL`.
14. Change `sales_team.auth_member` from ON DELETE CASCADE to SET NULL or RESTRICT. Deleting a user account must not delete sales history.
15. Drop the duplicate `fk_location` on `bonus.location_id`; keep one FK with one deliberate delete rule.
16. Make `bonus.location_id`, `individual_bonus.staff_id / sales_team_id` NOT NULL, and add `bonus_config_history_id` to `individual_bonus` so every payout carries its rate provenance.
17. Replace `staff.location_id bigint[]` and `sales_team.members bigint[]` with proper join tables so referential integrity and "staff belongs to this clinic" become enforceable.

## Fix the lifecycle

18. Add `status` to `orders` (`completed` / `voided` / `refunded` / `partially_refunded`) and stop hard-deleting sales.
19. Make the bonus roll-up net of returns: subtract `returns` / `return_qty` from `total_sales` and `bonus_sales`.
20. Add a recompute-for-date routine so a refund on a past day can correct that day's `bonus` and issue an adjusting `individual_bonus` row rather than editing history in place.
21. Block reversal of a bonus already marked paid; require an explicit adjustment instead.

## Fix authorization and operations

22. Add `getUser()` plus a role/permission check to `/api/bonuses/save` and `/api/bonuses/update-paid`, and a location-membership check to both sales-team routes.
23. Record who approved and who paid: `approved_by`, `approved_at`, `paid_by` on `bonus` and `individual_bonus`.
24. Fix `/api/bonuses/update-paid` to match teams by bonus date overlap (`valid_from <= date AND (valid_to IS NULL OR valid_to > date)`), not `valid_from = date`, and include the seller's rows.
25. Move the cron jobs to after the clinic day closes in Central time, and make the roll-up operate on a Central-time business date rather than UTC `CURRENT_DATE`.

26. Add an idempotency key to `POST /api/orders` and wrap its five writes in one transaction.
  27. Delete `trigger_distribute_individual_bonus()`, `sales_team.bonus_amount_individual`, and the dead branch in `bonuses/save`.
- 

## Verification notes

---

Everything above was read from source or queried live against `reiogvwjunkdrbhwewv`. Production was not touched.

The POS staff picker was checked: `components/POS/PosFields.tsx:51-54` loads staff with `.contains("location_id", [String(locationId)])`, so the list is location-scoped, and `toggleSelect` (190-193) tests membership by id, so the same person cannot be picked twice. Nothing re-validates the selection against the location at save time, client or server, and if the POS location changes while the modal stays open the stale selection is not cleared.

**The seller is counted twice.** `PosFields.tsx:278-283` posts only the picked staff as `members`, and `app/api/sales-team/route.ts:46` sets `auth_member: user.id` separately. The distribution function then does `member_count := array_length(members,1) + 1` when `auth_member` is set. So if the logged-in seller is also picked in the staff list — nothing prevents it — they receive two rows and the pool is split one way too many.

Explicitly **not verified from the existing implementation**:

- Authorization on marking a product `bonus_eligible` (`products` RLS was not examined in depth).
- Whether `staff.location_id` is stored as `text[]` or `int[]` in production. The POS query casts with `String(locationId)`; if the column is `int[]` the `contains` match returns zero rows silently and the picker shows an empty list rather than an error.
- Behaviour of the three cron jobs under failure or overlap. `cron.job_run_details` was not examined.

## Display-layer defects worth fixing while you are in there

- `bonus/individual/page.tsx:293-299` — "Highest Bonus Staff on Location" builds a per-staff tally, then discards it. `highestAmount` is the literal `0` and is never reassigned, so the card always renders `$0.00`.
- `bonus/location/fetch.ts:24-36` — `fetchPaidBonusesForDate(selectedDate)` computes a date range and then ignores both it and the argument, returning **every paid row in the table**, unbounded.
- `bonus/location/page.tsx:568-576` — `getTransactionStatus` and `getPaymentMethod` return `Math.random()` picks from hardcoded lists, rendered at 1788/1793 as if real. Currently unreachable because the sheet can never open, but it is one `setIsSheetOpen(true)` away.
- `bonus/location/page.tsx:1690, 1714` — paid historical rows fall back to the *current* config and to `computeBonusAmount` using *today's* totals, so old payouts can display the wrong rate.
- `bonus/location/page.tsx:536-556` — fires up to 10 concurrent `config-query` requests on every tab or date change purely to `console.log` the responses.
- The "Bonus eligibility" filter in `BonusFilterSheet.tsx:109-121` is collected into state and never applied; `filteredPatients` never reads it.
- `bonus/location/page.tsx:43-49` — `getTodayYMD` / `getYesterdayYMD` use `toISOString()`, i.e. UTC, so the default date flips in the early evening Central.